

Full Stack Development – 2
(Skill Enhanced Course)

23CS5S13

List of Experiments:

1. ExpressJS – Routing, HTTP Methods, Middleware. .

- a. Write a program to define a route, Handling Routes, Route Parameters, Query Parameters and URL building.
- b. Write a program to accept data, retrieve data and delete a specified resource using http methods.
- c. Write a program to show the working of middleware.

2. ExpressJS – Templating, Form Data .

- a. Write a program using templating engine.
- b. Write a program to work with form data.

3. ExpressJS – Cookies, Sessions, Authentication .

- a. Write a program for session management using cookies and sessions.
- b. Write a program for user authentication.

4. ExpressJS – Database, RESTful APIs .

- a. Write a program to connect MongoDB database using Mongoose and perform CRUD operations.
- b. Write a program to develop a single page application using RESTful APIs.

5. ReactJS – Render HTML, JSX, Components – function & Class .

- a. Write a program to render HTML to a web page.
- b. Write a program for writing markup with JSX.
- c. Write a program for creating and nesting components (function and class).

6. ReactJS – Props and States, Styles, Respond to Events .

- a. Write a program to work with props and states.
- b. Write a program to add styles (CSS & Sass Styling) and display data.
- c. Write a program for responding to events.

7. ReactJS – Conditional Rendering, Rendering Lists, React Forms .

- a. Write a program for conditional rendering.
- b. Write a program for rendering lists.
- c. Write a program for working with different form fields using react forms.

8. ReactJS – React Router, Updating the Screen .

- a. Write a program for routing to different pages using react router.
- b. Write a program for updating the screen.

9. ReactJS – Hooks, Sharing data between Components .

- a. Write a program to understand the importance of using hooks.
- b. Write a program for sharing data between components.

10. MongoDB – Installation, Configuration, CRUD operations .

- a. Install MongoDB and configure ATLAS
- b. Write MongoDB queries to perform CRUD operations on document using insert(), find(), update(), remove()

11. MongoDB – Databases, Collections and Records .

- a. Write MongoDB queries to Create and drop databases and collections.
- b. Write MongoDB queries to work with records using find(), limit(), sort(), createIndex(), aggregate().

12. Augmented Programs: (Any 2 must be completed) .

- a. Design a to-do list application using NodeJS and ExpressJS.
 - b. Design a Quiz app using ReactJS. b. Complete the MongoDB certification from MongoDB University website.
-

Experiment 1: ExpressJS – Routing, HTTP Methods, Middleware

a. Route Handling and URL Parameters

Aim: To implement routing in ExpressJS with support for path parameters and query strings.

Description: Demonstrates routing structure in Express, and how to extract dynamic values from the URL.

Step-by-Step Process:

1. Initialize NodeJS app with Express: *npm init -y* and *npm install express*
2. Create an Express server with GET route */hello/:name*
3. Add a route to handle query parameters */search?term=value*

Source Code:

```
const express = require('express');
const app = express();

app.get('/hello/:name', (req, res) => {
  res.send(`Hello, ${req.params.name}!`);
});

app.get('/search', (req, res) => {
  const term = req.query.term;
  res.send(`Search term: ${term}`);
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

Output:

- Accessing */hello/John* displays “Hello, John!”
- Accessing */search?term=nodejs* displays “Search term: nodejs”

b. HTTP Methods: Accept, Retrieve, and Delete Data

Aim: To perform basic CRUD using HTTP methods GET, POST, and DELETE in ExpressJS.

Description: Create routes to add and remove resources.

Step-by-Step Process:

1. Use *body-parser* middleware for POST body parsing.
2. Maintain an in-memory array as a database.
3. Create POST and DELETE endpoints.

Source Code:

```
const express = require('express');
const bodyParser = require('body-parser');
```

```

const app = express();
app.use(bodyParser.json());

let items = [];

app.post('/item', (req, res) => {
  items.push(req.body.name);
  res.send('Item added');
});

app.get('/items', (req, res) => {
  res.json(items);
});

app.delete('/item/:name', (req, res) => {
  items = items.filter(i => i !== req.params.name);
  res.send('Item deleted');
});

app.listen(3000);

```

Output:

- POST `/item` with JSON `{ "name": "Book" }` adds “Book” to items
- GET `/items` returns `["Book"]`
- DELETE `/item/Book` removes it from the array

c. Middleware Demonstration

Aim: To understand how middleware functions are used in ExpressJS.

Description: Show logging middleware and how it intercepts requests.

Step-by-Step Process:

1. Define a middleware that logs method and path.
2. Apply it globally before routes.

Source Code:

```

const express = require('express');
const app = express();

app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
});

app.get('/', (req, res) => res.send('Middleware demo'));

app.listen(3000);

```

Output:

- Console logs: *GET /* whenever the route is accessed

Experiment 2: ExpressJS – Templating, Form Data

a. Using Templating Engine

Aim: To develop a dynamic web page using a templating engine like EJS in ExpressJS.

Description: This experiment illustrates how to integrate EJS into an Express application to generate HTML dynamically.

Step-by-Step Process:

1. Initialize project: *npm init -y*
2. Install express and ejs: *npm install express ejs*
3. Set view engine to EJS in app.js
4. Create an EJS template in the views directory
5. Pass data from Express route to EJS for rendering

Source Code:

```
const express = require('express');
const app = express();

app.set('view engine', 'ejs');

app.get('/', (req, res) => {
  const user = { name: "Alice", age: 22 };
  res.render('profile', { user });
});

app.listen(3000);
```

views/profile.ejs:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>User Profile</h1>
    <p>Name: <%= user.name %></p>
    <p>Age: <%= user.age %></p>
  </body>
</html>
```

Output:

- Accessing */* renders an HTML page with the user details.

b. Handling Form Data

Aim: To demonstrate form handling and data capture using ExpressJS.

Description: Captures input from an HTML form and displays it on another page using EJS.

Step-by-Step Process:

1. Install body-parser: *npm install body-parser*
2. Create an HTML form using EJS
3. Handle POST request in Express

Source Code:

```
const express = require('express');
const bodyParser = require('body-parser');
const app = express();

app.set('view engine', 'ejs');
app.use(bodyParser.urlencoded({ extended: true }));

app.get('/', (req, res) => {
  res.render('form');
});

app.post('/submit', (req, res) => {
  const name = req.body.name;
  res.render('result', { name });
});

app.listen(3000);
```

views/form.ejs:

```
<form action="/submit" method="POST">
  Name: <input type="text" name="name">
  <button type="submit">Submit</button>
</form>
```

views/result.ejs:

```
<h2>Hello, <%= name %>!</h2>
```

Output:

- Form displays on root URL
- Submitted name is rendered as “Hello, [name]!”

Experiment 3: ExpressJS – Cookies, Sessions, Authentication

a. Session Management Using Cookies and Sessions

Aim: To demonstrate how to manage sessions in an ExpressJS application using cookies and session middleware.

Description: This experiment sets up session tracking using *express-session* and cookies to identify user sessions.

Step-by-Step Process:

1. Install necessary modules: *npm install express express-session*
2. Configure the *express-session* middleware.
3. Set up login and logout routes with session data.

Source Code:

```
const express = require('express');
const session = require('express-session');
const app = express();

app.use(session({
  secret: 'secretKey',
  resave: false,
  saveUninitialized: true,
  cookie: { maxAge: 60000 }
}));

app.get('/login', (req, res) => {
  req.session.username = 'testuser';
  res.send('User logged in');
});

app.get('/dashboard', (req, res) => {
  if (req.session.username) {
    res.send(`Welcome ${req.session.username}`);
  } else {
    res.send('Please log in first.');
```

Output:

- Visiting */login* creates a session.

- `/dashboard` displays the session username.
- `/logout` destroys the session.

b. User Authentication

Aim: To implement basic user authentication in ExpressJS.

Description: Authenticates users based on predefined credentials and uses sessions to maintain login state.

Step-by-Step Process:

1. Set up a login form route and a POST handler.
2. Validate submitted credentials.
3. Maintain session on successful login.

Source Code:

```
const express = require('express');
const bodyParser = require('body-parser');
const session = require('express-session');
const app = express();

app.use(bodyParser.urlencoded({ extended: true }));
app.use(session({
  secret: 'authSecret',
  resave: false,
  saveUninitialized: true
}));

const users = { 'admin': '1234' };

app.get('/login', (req, res) => {
  res.send('<form method="post"><input name="username"/><input name="password" type="password"/><button>Login</button></form>');
});

app.post('/login', (req, res) => {
  const { username, password } = req.body;
  if (users[username] && users[username] === password) {
    req.session.user = username;
    res.send('Login successful');
  } else {
    res.send('Invalid credentials');
  }
});

app.get('/home', (req, res) => {
  if (req.session.user) {
    res.send(`Welcome, ${req.session.user}`);
  } else {

```



```
    res.send('Not authenticated');
  }
});

app.listen(3000);
```

Output:

- User submits login form with correct credentials.
- If correct, receives “Login successful” and session persists.
- Visiting */home* displays the logged-in user’s name.

Experiment 4: ExpressJS – Database, RESTful APIs

a. Connect to MongoDB Using Mongoose and Perform CRUD Operations

Aim: To connect an ExpressJS backend to a MongoDB database using Mongoose and implement CRUD operations.

Description: This experiment demonstrates how to use Mongoose to define a schema, connect to MongoDB, and perform Create, Read, Update, and Delete operations on documents.

Step-by-Step Process:

1. Create a new Node project and install dependencies:

```
npm init -y
npm install express mongoose body-parser
```

2. Create a model using Mongoose schema.
3. Set up routes to handle CRUD operations.
4. Test endpoints using Postman or any REST client.

Source Code:

```
const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');

const app = express();
app.use(bodyParser.json());

mongoose.connect('mongodb://localhost:27017/express_crud', {
  useNewUrlParser: true,
  useUnifiedTopology: true
});

const Item = mongoose.model('Item', { name: String });
```

```

// Create
app.post('/items', async (req, res) => {
  const item = new Item({ name: req.body.name });
  await item.save();
  res.send('Item saved');
});

// Read
app.get('/items', async (req, res) => {
  const items = await Item.find();
  res.json(items);
});

// Update
app.put('/items/:id', async (req, res) => {
  await Item.findByIdAndUpdate(req.params.id, { name: req.body.name });
  res.send('Item updated');
});

// Delete
app.delete('/items/:id', async (req, res) => {
  await Item.findByIdAndDelete(req.params.id);
  res.send('Item deleted');
});

app.listen(3000, () => console.log('Server running on port 3000'));

```

Output:

- *POST* /items with JSON { "name": "Book" } saves a new item.
- *GET* /items returns the list of items in the database.
- *PUT* /items/:id updates a specific item.
- *DELETE* /items/:id deletes a specific item.

b. Build a Single Page Application Using RESTful APIs

Aim: To create a single-page web application that consumes RESTful APIs created in ExpressJS.

Description: Use frontend (HTML/JavaScript) to make API calls to the backend for managing data in MongoDB.

Step-by-Step Process:

1. Ensure RESTful API from part (a) is running.
2. Create a frontend page using HTML and JavaScript (Fetch API).
3. Implement *GET*, *POST*, and *DELETE* operations in the frontend.

Source Code (Frontend - index.html):

```

<!DOCTYPE html>
<html>
<body>
  <h2>Items</h2>
  <input id="itemInput" placeholder="Enter item">
  <button onclick="addItem()">Add</button>
  <ul id="itemList"></ul>

  <script>
    async function fetchItems() {
      const res = await fetch('/items');
      const items = await res.json();
      document.getElementById('itemList').innerHTML =
        items.map(i => `<li>${i.name} <button onclick="deleteItem('${i._id}')">Delete</button></li>`).join("");
    }

    async function addItem() {
      const name = document.getElementById('itemInput').value;
      await fetch('/items', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ name })
      });
      fetchItems();
    }

    async function deleteItem(id) {
      await fetch(`/items/${id}`, { method: 'DELETE' });
      fetchItems();
    }

    fetchItems();
  </script>
</body>
</html>

```

Output:

- User can add an item using input and button.
- Items are displayed in a list.
- Clicking “Delete” removes the item from MongoDB and refreshes the list.

Experiment 5: ReactJS – Render HTML, JSX, Components – Function & Class

a. Render HTML to a Web Page

Aim: To render static HTML content inside a React component.

Description: Demonstrates the use of JSX to embed HTML in a React application.

Step-by-Step Process:

1. Create a new React application: `npx create-react-app myapp`
2. Replace the contents of `App.js` with a simple HTML layout.
3. Start the development server with `npm start`

Source Code (``):

```
import React from 'react';

function App() {
  return (
    <div>
      <h1>Welcome to React</h1>
      <p>This is rendered using JSX.</p>
    </div>
  );
}

export default App;
```

Output:

- Webpage displays a heading “Welcome to React” and a paragraph.

b. Writing Markup with JSX

Aim: To use JSX syntax to create component UI structure.

Description: JSX lets you write HTML tags directly inside JavaScript code.

Step-by-Step Process:

1. Add expressions and attributes inside JSX.
2. Understand differences between HTML and JSX (e.g., `className`, `htmlFor`).

Source Code:

```
function Greeting() {
  const name = 'Alice';
  return (
    <div>
      <h2>Hello, {name}!</h2>
    </div>
  );
}

export default Greeting;
```

Output:

- Displays: “Hello, Alice!”

c. Creating and Nesting Components (Function and Class)

Aim: To create both function and class components in React and nest them.

Description: Illustrates reusability of code by nesting multiple components.

Step-by-Step Process:

1. Define two components: one using function, one using class.
2. Use function component inside class component (or vice versa).
3. Import components as needed.

Source Code:

```
import React from 'react';

function Header() {
  return <h1>This is a Header</h1>;
}

class App extends React.Component {
  render() {
    return (
      <div>
        <Header />
        <p>This is a class-based component</p>
      </div>
    );
  }
}

export default App;
```

Output:

- Displays: “This is a Header” followed by “This is a class-based component”

Experiment 6: ReactJS – Props and States, Styles, Respond to Events

a. Working with Props and State

Aim: To demonstrate how to pass data through props and manage component-specific state using the *useState* hook.

Description: In this experiment, a parent component passes props to a child, while the child maintains internal state and reacts to user input.

Step-by-Step Process:

1. Create a new React app.
2. Define a parent component that passes data as props.
3. Define a child component that displays props and uses state.

Source Code:

```
// ParentComponent.js
import React from 'react';
import Child from './Child';

function ParentComponent() {
  return <Child name="Alice" />;
}

export default ParentComponent;

// Child.js
import React, { useState } from 'react';

function Child({ name }) {
  const [age, setAge] = useState(20);

  return (
    <div>
      <h1>Hello, {name}</h1>
      <p>Age: {age}</p>
      <button onClick={() => setAge(age + 1)}>Increase Age</button>
    </div>
  );
}

export default Child;
```

Output:

- Renders “Hello, Alice” with initial age.
- Clicking the button increases the displayed age.

b. Add Styles (CSS & SASS Styling) and Display Data

Aim: To apply inline styles and external CSS/SASS styling in a React component.

Description: Use both inline and external styles to enhance visual display of React content.

Step-by-Step Process:

1. Create a CSS or SCSS file and import it.
2. Apply both inline and class-based styles in JSX.

Source Code:

```
/* styles.css */
.title {
  color: blue;
  font-size: 24px;
```

```

    margin-bottom: 10px;
  }

// StyledComponent.js
import React from 'react';
import './styles.css';

function StyledComponent() {
  const inlineStyle = { color: 'green', fontWeight: 'bold' };

  return (
    <div>
      <h1 className="title">Styled using CSS</h1>
      <p style={inlineStyle}>This paragraph uses inline style</p>
    </div>
  );
}

export default StyledComponent;

```

Output:

- Header in blue from external CSS.
- Paragraph in green bold font via inline style.

c. Responding to Events

Aim: To show how to handle user-triggered events (e.g., button clicks) in React.

Description: Create interactive UI by capturing click, change, and submit events in components.

Step-by-Step Process:

1. Use *onClick*, *onChange*, and *onSubmit* attributes in JSX.
2. Define handler functions in components.

Source Code:

```

import React, { useState } from 'react';

function EventDemo() {
  const [message, setMessage] = useState('Hello!');

  const handleClick = () => setMessage('Button Clicked!');

  return (
    <div>
      <p>{message}</p>
      <button onClick={handleClick}>Click Me</button>
    </div>
  );
}

```

```
export default EventDemo;
```

Output:

- Initially displays “Hello!”
- Clicking button changes it to “Button Clicked!”

Experiment 7: ReactJS – Conditional Rendering, Rendering Lists, React Forms

a. Conditional Rendering

Aim: To demonstrate conditional rendering of components using ReactJS.

Description: Conditional rendering allows components to display different UI based on certain conditions (e.g., logged in vs. not logged in).

Step-by-Step Process:

1. Create a component and use *useState* to toggle a boolean flag.
2. Use JavaScript conditional (*if*, ternary operator) to render different content.

Source Code:

```
import React, { useState } from 'react';

function ConditionalComponent() {
  const [isLoggedIn, setIsLoggedIn] = useState(false);

  return (
    <div>
      {isLoggedIn ? <h2>Welcome Back!</h2> : <h2>Please Log In</h2>}
      <button onClick={() => setIsLoggedIn(!isLoggedIn)}>
        {isLoggedIn ? 'Logout' : 'Login'}
      </button>
    </div>
  );
}

export default ConditionalComponent;
```

Output:

- Displays “Please Log In” initially.
- Clicking “Login” changes view to “Welcome Back!” and vice versa.

b. Rendering Lists

Aim: To render lists dynamically using the *map()* function in React.

Description: This experiment illustrates how to render an array of data in a component and handle keys.

Step-by-Step Process:

1. Define a data array inside the component.
2. Use `map()` function to iterate over array.
3. Render list items and use a unique key.

Source Code:

```
import React from 'react';

function ListComponent() {
  const fruits = ['Apple', 'Banana', 'Cherry'];

  return (
    <ul>
      {fruits.map((fruit, index) => (
        <li key={index}>{fruit}</li>
      ))}
    </ul>
  );
}
```

```
export default ListComponent;
```

Output:

- Displays a bulleted list: Apple, Banana, Cherry.

c. React Forms

Aim: To capture user input using controlled components (React Forms).

Description: React forms use state to capture form field values. This example collects name and displays it on submission.

Step-by-Step Process:

1. Create form with input field and a submit button.
2. Use `useState` to store input value.
3. Display the entered value on form submission.

Source Code:

```
import React, { useState } from 'react';

function FormComponent() {
  const [name, setName] = useState("");
  const [submittedName, setSubmittedName] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    setSubmittedName(name);
  };
}
```

```

    };

    return (
      <div>
        <form onSubmit={handleSubmit}>
          <label>Name: </label>
          <input value={name} onChange={(e) => setName(e.target.value)} />
          <button type="submit">Submit</button>
        </form>
        {submittedName && <h2>Hello, {submittedName}</h2>}
      </div>
    );
  }
}

export default FormComponent;

```

Output:

- User types name and clicks Submit.
- Displays “Hello, [name]” below the form.

Experiment 8: ReactJS – React Router, Updating the Screen

a. Routing to Different Pages Using React Router

Aim: Implement routing between pages using React Router.

Description: Demonstrates multi-page navigation using react-router-dom.

Step-by-Step Process:

1. Install React Router: npm install react-router-dom
2. Create components for different pages.
3. Configure routes in App.js using BrowserRouter, Routes, and Route.

Source Code:

```

import React from 'react';
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function Home() {
  return <h2>Home Page</h2>;
}

function About() {
  return <h2>About Page</h2>;
}

function App() {
  return (
    <BrowserRouter>
      <nav>

```

```

    <Link to="/">Home</Link> | <Link to="/about">About</Link>
  </nav>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
  </Routes>
</BrowserRouter>
);
}

```

export default App;

Output:

- Navigation links route between Home and About pages.

b. Updating the Screen

Aim: Dynamically update UI using state changes.

Description: Uses useState to re-render component based on user interaction.

Step-by-Step Process:

4. Initialize state with useState.
5. Trigger state update using button click.
6. Display updated value in JSX.

Source Code:

```

import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}

export default Counter;

```

Output:

- Clicking the button increases the count.

Experiment 9: ReactJS – Hooks, Sharing Data Between Components

a. Using Hooks

Aim: Utilize React hooks to manage state and lifecycle features.

Description: Demonstrates useState and useEffect in functional components.

Step-by-Step Process:

7. Use useState to manage count.
8. Use useEffect to log value changes.

Source Code:

```
import React, { useState, useEffect } from 'react';

function HookDemo() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log("Count updated:", count);
  }, [count]);

  return (
    <div>
      <button onClick={() => setCount(count + 1)}>Click {count}</button>
    </div>
  );
}

export default HookDemo;
```

Output:

- Logs value changes and updates button count on click.

b. Sharing Data Between Components

Aim: Pass and update data between parent and child components.

Description: Uses props and callbacks to share and mutate state.

Step-by-Step Process:

9. Maintain state in parent component.
10. Pass state and handler as props to child.
11. Trigger state change in parent from child.

Source Code:

```
// ParentComponent.js
import React, { useState } from 'react';
import Child from './Child';

function ParentComponent() {
  const [message, setMessage] = useState("Initial Message");
```

```

const updateMessage = () => setMessage("Updated from Child");

return (
  <div>
    <h2>{message}</h2>
    <Child updateMessage={updateMessage} />
  </div>
);
}

export default ParentComponent;

// Child.js
import React from 'react';

function Child({ updateMessage }) {
  return <button onClick={updateMessage}>Update Parent</button>;
}

export default Child;

```

Output:

- Clicking child button updates parent component message.

Experiment 10: MongoDB – Installation, Configuration, CRUD Operations

a. Install MongoDB and Configure Atlas

Aim: To install MongoDB and configure a cloud database using MongoDB Atlas.

Description: Guides through MongoDB setup, account creation, and cluster configuration.

Step-by-Step Process:

12. Go to <https://www.mongodb.com/cloud/atlas>
13. Create account and new cluster.
14. Add database user and IP whitelist.
15. Connect using Mongo URI from Atlas dashboard.

Output:

- A fully functional cloud database ready for connection from Express or Mongoose.

b. CRUD Operations Using insert(), find(), update(), remove()

Aim: Perform CRUD operations on MongoDB collection using Node.js.

Source Code:

```

const { MongoClient } = require('mongodb');

```

```

const uri = 'your-mongodb-uri';
const client = new MongoClient(uri);

async function run() {
  try {
    await client.connect();
    const db = client.db('labdb');
    const collection = db.collection('students');

    await collection.insertOne({ name: 'John', age: 22 });
    const docs = await collection.find().toArray();
    console.log('Find:', docs);
    await collection.updateOne({ name: 'John' }, { $set: { age: 23 } });
    await collection.deleteOne({ name: 'John' });
  } finally {
    await client.close();
  }
}

run();

```

Output:

- Logs inserted, updated, and deleted documents in terminal.

Experiment 11: MongoDB – Databases, Collections and Records

a. Create and Drop Databases and Collections

Aim: Manage MongoDB databases and collections.

Step-by-Step Process:

16. Use MongoDB shell or Compass.

17. Commands:

use mydb

use mydb

3. Create collection:

db.createCollection("products")

4. Drop collection:

db.products.drop()

5. Drop database:

db.dropDatabase()

Output:

- Database and collection created and removed.

b. Work with Records using find(), limit(), sort(), createIndex(), aggregate()

Aim: Query and analyze data using MongoDB commands.

Source Code:

```
// Insert sample data
db.products.insertMany([
  { name: 'Pen', price: 10 },
  { name: 'Pencil', price: 5 },
  { name: 'Eraser', price: 7 }
])

// Find all
db.products.find()

// Limit results
db.products.find().limit(2)

// Sort by price
db.products.find().sort({ price: -1 })

// Create index
db.products.createIndex({ name: 1 })

// Aggregate
db.products.aggregate([
  { $group: { _id: null, total: { $sum: "$price" } } }
])
```

Output:

Shows queries, limits, sorting, index, and total aggregation

Experiment 12: Augmented Programs

a. To-Do List Application using NodeJS and ExpressJS

Aim: To build a simple To-Do list application with backend support using NodeJS and ExpressJS.

Description: This application allows users to add, view, and delete tasks. Data is stored temporarily in memory (or can be extended to MongoDB).

Step-by-Step Process:

1. Initialize a new Node project:

```
mkdir todo-app && cd todo-app
npm init -y
npm install express body-parser ejs
```

2. Create *app.js*, configure Express, and set up routes.

3. Use EJS as the templating engine.
4. Store task data in an array or connect to MongoDB for persistence.

Source Code (app.js):

```
const express = require("express");
const bodyParser = require("body-parser");
const app = express();

app.set("view engine", "ejs");
app.use(bodyParser.urlencoded({ extended: true }));

let tasks = [];

app.get("/", (req, res) => {
  res.render("index", { tasks });
});

app.post("/add", (req, res) => {
  const task = req.body.task;
  if (task) tasks.push(task);
  res.redirect("/");
});

app.post("/delete", (req, res) => {
  const taskToDelete = req.body.task;
  tasks = tasks.filter(t => t !== taskToDelete);
  res.redirect("/");
});

app.listen(3000, () => console.log("Server running on port 3000"));
```

EJS Template (views/index.ejs):

```
<!DOCTYPE html>
<html>
  <body>
    <h2>To-Do List</h2>
    <form action="/add" method="post">
      <input name="task" required />
      <button type="submit">Add</button>
    </form>
    <ul>
      <% tasks.forEach(task => { %>
        <li>
          <form action="/delete" method="post" style="display:inline">
            <%= task %>
            <input type="hidden" name="task" value="<%= task %>" />
            <button type="submit">Delete</button>
          </form>
        </li>
      <% } %>
    </ul>
  </body>
</html>
```



```
<% }) %>
</ul>
</body>
</html>
```

Output:

- Homepage displays form and task list.
 - Users can add a task using the form.
 - Tasks appear in list format with a delete button.
 - Clicking delete removes task from the list.
-

b. Quiz App using ReactJS

Aim: To develop an interactive quiz application using ReactJS with options, scoring, and result display.

Description: This app presents a series of questions with multiple choices. Users select answers and view scores at the end.

Step-by-Step Process:

1. Create a new React app: `npx create-react-app quiz-app`
2. Create a `Quiz.js` component to handle quiz logic.
3. Maintain questions in an array of objects.
4. Display one question at a time and update score.

Source Code (Quiz.js):

```
import React, { useState } from 'react';

const questions = [
  { q: "Capital of India?", options: ["Delhi", "Mumbai", "Kolkata"], answer: "Delhi" },
  { q: "2 + 2?", options: ["3", "4", "5"], answer: "4" }
];

function Quiz() {
  const [index, setIndex] = useState(0);
  const [score, setScore] = useState(0);
  const [showResult, setShowResult] = useState(false);

  const handleOption = (selected) => {
    if (selected === questions[index].answer) setScore(score + 1);
    const next = index + 1;
    if (next < questions.length) setIndex(next);
    else setShowResult(true);
  };

  return (
```

```
<div>
  {!showResult ? (
    <div>
      <h3>{questions[index].q}</h3>
      {questions[index].options.map((opt, i) => (
        <button key={i} onClick={() => handleOption(opt)}>{opt}</button>
      ))}
    </div>
  ) : (
    <h2>Your Score: {score}/{questions.length}</h2>
  )}
</div>
);
}
```

export default Quiz;

Output:

- Displays questions one-by-one.
 - User selects an answer → next question loads.
 - Final score shown after last question.
-